

Programming Assignment 1: Frozen Lake from First Principles Using Q-Learning

Name: Akorlie Edward Tsatsu

Course: DSCD 614 – Reinforcement Learning

Department: Computer Science, University of Ghana

Semester: II, 2025/2026 Academic Year

Index Number: 22424530

1. Introduction

Reinforcement Learning (RL) is a branch of machine learning in which an agent learns to make sequential decisions by interacting with an environment. Unlike supervised learning, which relies on labelled training data, an RL agent receives feedback in the form of numerical rewards and must discover, through trial and error, which actions lead to the greatest cumulative reward over time. The theoretical foundation of modern RL rests on the Markov Decision Process (MDP) framework, which formally describes the relationship between states, actions, transition probabilities, and rewards.

The Frozen Lake problem is a classic RL benchmark that captures the core challenge of sequential decision-making under uncertainty. An agent must navigate an 8×8 grid from a fixed start position to a goal position while avoiding holes in the ice. The environment is deterministic: each action reliably moves the agent in the intended direction, but the grid layout forces the agent to chart a safe path through a field of hazards. The simplicity of the state and action spaces makes Frozen Lake an ideal testbed for validating a Q-Learning implementation from first principles, where every component (the environment, the agent, the training loop, and the evaluation procedure) must be written entirely in Python without the aid of any RL framework.

2. Environment Design

2.1 Grid Layout

The 8×8 Frozen Lake map used in this assignment is:

```
S F F F F F F F
F F F F F F F F
F F F H F F F F
F F F H F F F F
F F F H F F F F
F H H F F F H F
F H F F H F H F
F F F H F F F G
```

Each cell belongs to one of four types: **S** (Start), **F** (Frozen, safe), **H** (Hole, terminal), and **G** (Goal, terminal). The agent begins at S (top-left) and must reach G (bottom-right).

2.2 State Representation

The environment encodes each cell as a flat integer state: $s = \text{row} \times 8 + \text{col}$, giving 64 unique states numbered 0–63. State 0 corresponds to the start position (0, 0) and state 63 to the goal

(7, 7). This flat encoding maps directly to rows and columns of the Q-table, enabling efficient $O(1)$ lookup.

2.3 Action Representation

The agent may take one of four actions at each time step:

Code	Direction	Effect
0	Left	col - 1
1	Down	row + 1
2	Right	col + 1
3	Up	row - 1

Moves that would carry the agent beyond the grid boundary are clamped; the agent remains in its current cell.

2.4 Reward Structure

Event	Reward
Reach Goal (G)	+1.0
Fall into Hole (H)	0.0
Any other step	0.0

An episode terminates immediately upon reaching either G or H, or after a maximum of 200 steps (a safety cap to prevent infinite loops during training). The sparse reward structure, positive only at the goal, makes learning challenging and tests the agent's ability to propagate value estimates back through many steps.

3. Q-Learning Algorithm

3.1 Description

Q-Learning is a model-free, off-policy temporal difference (TD) algorithm introduced by Watkins (1989). It maintains a Q-table (a matrix $Q(s, a)$ of expected cumulative discounted rewards for every (state, action) pair) and updates these estimates incrementally as the agent interacts with the environment. Over sufficiently many episodes, the Q-table converges toward the optimal action-value function Q^* , from which the optimal policy π^* can be directly read off as the greedy action at each state.

The algorithm is *off-policy* because the update target uses the maximum Q-value over next-state actions regardless of the action the agent actually takes, allowing it to learn the optimal policy while following an exploratory behaviour policy.

3.2 Update Equation

After every step, the Q-table is updated according to:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

where:

- α (learning rate): controls how much each new estimate overrides the old one
- γ (discount factor): weights future rewards relative to immediate ones; $\gamma = 0.99$ encourages long-horizon planning
- r : the immediate reward received after taking action a in state s
- s' : the next state
- $\max_{a'} Q(s', a')$: the best possible future value from s'

When an episode terminates (`done = True`), the bootstrap term is dropped and the target reduces to r alone, ensuring the Q -values of terminal states are not updated with erroneous future estimates.

3.3 Exploration Strategy

A pure greedy policy (always taking the action with the highest Q -value) would fail early in training because the Q -table is initialised to zero; every action appears equally good, and the agent would exploit a randomly selected action forever. To ensure adequate exploration, an **ϵ -greedy strategy** is used:

- With probability ϵ , the agent selects a random action (explore)
- With probability $1 - \epsilon$, the agent selects $\text{argmax } Q[\text{state}]$ (exploit)

To handle ties in Q -values (common early in training when Q -table rows are all zero), the agent randomly selects uniformly among all actions achieving the maximum value rather than deterministically returning index 0. This prevents the agent from locking into a degenerate left-biased policy before any learning has occurred.

Epsilon decay transitions the agent from pure exploration at the start of training toward near-pure exploitation as the Q -table matures:

$$\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon \times \epsilon_{\text{decay}})$$

A decay rate of 0.9997 per episode was chosen to ensure ϵ remains above 0.01 for approximately the first 15,000 of 20,000 training episodes, giving the agent ample exploration time on this map.

4. Training Methodology

4.1 Hyperparameters

Parameter	Symbol	Value
Training episodes	-	20,000
Learning rate	α	0.1
Discount factor	γ	0.99
Initial epsilon	ϵ	1.0
Minimum epsilon	$\epsilon_{\min}$	0.01

Parameter	Symbol	Value
Epsilon decay rate	-	0.9997
Max steps per episode	-	200

4.2 Hyperparameter Experiments

Three independent configurations were trained and compared to justify the default hyperparameter selection:

Config	α	γ	ϵ_{decay}	Training Success Rate
fast-learn	0.3	0.95	0.9995	88.2%
slow-learn	0.05	0.99	0.9997	85.8%
default	0.1	0.99	0.9997	77.5%

Each configuration ran for 20,000 episodes with a fresh Q-table. The default configuration ($\alpha=0.1$, $\gamma=0.99$) was selected for the final policy because it balances learning stability (lower α avoids overshooting good estimates) with long-horizon foresight ($\gamma=0.99$). Although fast-learn achieves a higher training success rate, its shorter discount horizon ($\gamma=0.95$) may undervalue distant rewards. The default configuration produced a deterministic policy that achieves 100% success in greedy evaluation.

4.3 Training Loop

Each episode proceeds as follows:

1. Reset environment to state 0
2. While not terminal and steps < 200:
 - Select action via ϵ -greedy policy
 - Execute action; observe (s' , r , done)
 - Update Q-table: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$
 - Advance to s'
3. Decay ϵ after each episode

5. Experimental Results

5.1 Training Performance

Figure 1 shows three training curves for the default configuration over 20,000 episodes:

- **Left panel:** Rolling average reward (window = 500 episodes): rises from near zero and plateaus around 0.7–0.8, reflecting consistent goal-reaching.

- **Centre panel:** Rolling success rate (%): climbs steadily as the agent learns safe paths, stabilising above 70%.
- **Right panel:** Epsilon decay curve: exponential decay from 1.0 to near 0.01, showing the transition from exploration to exploitation.

(See *results/training_results.png*)

5.2 Greedy Evaluation

After training, the agent was evaluated over 1,000 episodes with $\epsilon = 0$ (pure greedy, no exploration):

Metric	Value
Episodes evaluated	1,000
Successful runs	1,000
Failures	0
Success rate	100.0%
Average reward	1.0000

The 100% greedy success rate demonstrates that the Q-table has converged to an optimal deterministic policy; the agent reliably navigates from start to goal across every evaluation episode.

6. Learned Policy

The policy extracted from the trained Q-table by taking the greedy action at each non-terminal state is:

↓	↓	↓	↓	↓	↓	↓	←	
→	→	→	→	↓	↓	←	←	
↑	↑	↑	H	↓	↓	↓	←	
↑	←	↑	H	→	↓	←	←	
↑	↑	↑	H	→	↓	←	←	
↑	H	H	→	→	↓	H	↑	
←	H	←	↑	H	↓	H	↓	
←	←	←	H	→	→	→	G	

The policy shows clear structure: the agent learns to funnel toward column 5 in the upper half, descend through the safe corridor on the right side of the map, and approach the goal (7,7) from the left. Hole cells are marked H and the goal is marked G. States adjacent to holes show defensive arrows directing the agent away from danger.

7. Challenges Encountered

Sparse rewards and exploration collapse. The most significant challenge was the combination of sparse rewards (goal reached on fewer than 0.3% of random-walk episodes) and an initially aggressive epsilon decay of 0.995. With this decay, epsilon reached its minimum (0.01) after only ~920 episodes, far too early for the agent to have gathered sufficient goal-reaching experience. The

Q-table remained at zero for most states, and as ϵ collapsed, the agent locked into always selecting action 0 (Left), producing a 0% success rate. The solution was to reduce ϵ_{decay} to 0.9997, maintaining meaningful exploration across the full 20,000-episode training run.

Greedy tie-breaking on a zero-initialised Q-table. Early in training, all Q-values are zero, making every action equally good. Python's `np.argmax` deterministically returns index 0 (Left) on ties, which caused the agent to push against the left wall repeatedly during the low- ϵ phase. This was fixed by sampling uniformly among all actions that achieve the maximum Q-value, ensuring diverse exploration even under near-zero epsilon.

No stochastic environment. The deterministic environment means the agent can learn a perfect policy without needing to account for transition uncertainty. This makes convergence tractable but also means the learned policy may not generalise to a stochastic version of the map without retraining.

8. Conclusion

This assignment implemented a complete Q-Learning solution for the Frozen Lake problem from first principles in Python, without any RL framework. The custom `FrozenLakeEnv` environment faithfully models the 8×8 grid, and the `QLearningAgent` implements the standard Q-Learning update rule with ϵ -greedy exploration and multiplicative epsilon decay. After 20,000 training episodes with carefully tuned hyperparameters ($\alpha=0.1$, $\gamma=0.99$, $\epsilon_{\text{decay}}=0.9997$), the agent converges to an optimal deterministic policy achieving **100% success** in greedy evaluation across 1,000 episodes. The key engineering challenges (exploration collapse and greedy tie-breaking on a zero-initialised table) were identified through empirical testing and resolved with targeted fixes, demonstrating the importance of hyperparameter sensitivity analysis in RL system design.